基于 SSD 的过滤向量检索系统性能测试与设计比较

计算机系统设计 · 课程实验报告

姓名：_____　　学号：_____　　日期：2026 年 9 月 30 日

1. 实验环境与复现信息

项目	内容
CPU	12th Gen Intel Core i7-12700H（WSL2 内可见 20 个逻辑处理器）
内存	宿主机约 15.7 GB；WSL2 配置为 12 GB，实验时 `free` 显示约 11 GiB
SSD 型号与文件系统	查询运行在 WSL2 的 ext4 虚拟磁盘上。该虚拟磁盘位于宿主机 WD PC SN810 NVMe SSD（约 512 GB，系统盘）。`lsblk` 把虚拟盘标成 ROTA=1，这是虚拟磁盘的报告方式，宿主机介质是 NVMe SSD
操作系统与内核	Ubuntu 22.04.3 LTS，内核 6.18.33.2-microsoft-standard-WSL2；宿主机 Windows 11
编译器	g++ 11.4.0（Ubuntu 11.4.0-1ubuntu1~22.04.3）
PipeANN-Filter Commit	`2ec84fe20c37dc3cc236b66f7eba31c0cc80bf1e`
GateANN Commit	`71e87cbf0e25c0400c0109dd25a64a7c4dccc9dd`
I/O Backend	AIO
公共参数	threads=1，beamwidth=8，K=10，mem_L=0，缓存预算=0；GateANN FullAdj neighbors=32
正式实验 experiment-id	`20260930-072449`

Debug 阶段两套系统都在 10 万向量上构建成功。执行 `scripts/run_debug_validation.sh` 后输出 `DEBUG CORRECTNESS CHECK PASS`。High 负载上 PipeANN-Filter（L=40）Recall@10 为 96.30%，GateANN（L=320）为 95.65%，查询数均为 200。Debug 运行标识为 `debug-check-20260930-071711`。正式实验使用课程提供的 100 万向量预构建索引，`download_indexes.sh` 与正式脚本均完成索引校验。

源码和课程脚本修改：无。GateANN 的一行 AIO 数组访问补丁由课程脚本 `prepare_gateann.sh` 自动应用，不改变算法和索引格式。

2. 系统设计理解

2.1 PipeANN-Filter

PipeANN 面向放在 SSD 上的图搜索。向量和 Vamana 邻接表按页放在 SSD 索引里，DRAM 中保留入口点、PQ 码本和压缩向量，用来在不每次读回完整向量的情况下估计距离。查询是对齐 SSD 的 best-first search：搜索预算 L 决定扩展多少候选。L 变大时，距离计算、逻辑 I/O、Recall 和延迟通常一起上升。

PipeANN-Filter 在这条路径上加入过滤，并在三条路径中做动态路由。源码 `src/search/spec_filter_search.cpp` 为 Pre-filter、In-filter、Post-filter 分别估计 SSD 读次数和距离比较次数，选估计代价最低的一条。Pre-filter 先按谓词取出候选再搜索，适合满足条件的点很少的查询；In-filter 在扩展图的过程中做近似成员判断，允许假阳性以换取少读盘；Post-filter 先搜索再过滤，可能把 I/O 花在最终会被丢掉的点上。预期收益是避开明显不合算的过滤时机。代价是路由估计可能选错，In-filter 的假阳性会带来额外 I/O 和比较。

2.2 GateANN

GateANN 同样把图和向量放在 SSD 页中，但把过滤判断尽量提前到读盘之前。本实验日志显示，100 万点的多标签矩阵 `SpmatFilterStore` 约 49 MB，放在 DRAM 中。Graph Tunneling 使用常驻内存的 FullAdj：正式索引以 mmap 加载 `yfcc.full_adj.R32`，约 0.123 GB，每个点最多 32 个邻居。

一次过滤查询先用 Pre-I/O Filtering 丢掉不满足谓词的邻居，因此这些点不会变成 SSD 读，对应指标是 FilterSkips。过滤会把原图切碎，Tunneling 再沿 FullAdj 跳到仍然满足谓词、且原本可能走不到的邻居，用来恢复可达性。预期收益是减少无效 SSD 读。代价是 FullAdj 和标签矩阵占用 DRAM，隧道遍历消耗 CPU；过滤很强而搜索预算不够时，I/O 已经很少，Recall 仍可能上不去。

2.3 设计比较

系统	过滤查询机制	DRAM 中的主要结构	SSD 中的主要结构	预期优势	潜在代价
PipeANN-Filter	按估计代价在 Pre / In / Post 三条路径间路由；In-filter 允许假阳性	入口点、PQ 码本与压缩向量、路由用的过滤状态	向量、Vamana 图和标签相关索引页	按选择率换路径，避免单一过滤时机	路由选错；In-filter 假阳性带来额外 I/O
GateANN	读盘前过滤，再用 FullAdj 做 Graph Tunneling	标签矩阵（约 49 MB）和 FullAdj（约 0.123 GB）	向量与磁盘图索引	跳过无效页，并补过滤后的图连通性	额外 DRAM；强过滤时隧道和预算可能仍不够

两套系统的差别主要在过滤发生的位置。PipeANN-Filter 把过滤嵌进搜索并动态换路径，图本身仍按原始邻接扩展，连通性较好，但 In-filter 会读到一些后来不算数的页。GateANN 把过滤放到 I/O 之前，用内存里的 FullAdj 补被剪断的边，所以逻辑 I/O 可以少很多，内存占用更高。索引体积则相反：本次测得 PipeANN-Filter 约 8.43 GiB，GateANN 约 486 MiB。

3. 实验方法

正式数据是 YFCC 100 万条 192 维 uint8 向量，距离为 L2。三档真实标签工作负载各 200 条查询。Selectivity 范围和中位数为：Low 0.250%–0.851%（0.545%），Medium 1.765%–4.995%（2.958%），High 7.074%–17.089%（12.323%）。High 上 PipeANN-Filter 的 L 为 20、40、80，GateANN 为 160、320、640。Selectivity 比较固定各自的代表性 L，即 PipeANN-Filter L=40、GateANN L=320。独立重复 3 次。表格和曲线使用 `system + bucket + L` 分组后的中位数，误差线为三轮最小值到最大值。延迟使用 Query Phase 的 `mean_latency_us`，不用包含加载时间的 `elapsed_seconds`。

两套系统的 L 不能直接比较。它都表示搜索预算，但数值尺度和同一次扩展所做的过滤、I/O 不同。课程给出的是各自校准过的三档预算，不是同一个 L。比较性能时要找 Recall 接近的运行点，同时看延迟和逻辑 I/O，不能把“L 更大”或“L 相同”当成公平。

实验期间没有修改课程统一参数，也没有执行 `drop_caches`。索引刚刚解压完，正式脚本开始时页缓存已经约有 10 GiB。三轮之间 Recall 和逻辑 I/O 几乎不变，延迟多轮下降，符合页缓存变热，而不是搜索路径在变。因此本文把 Recall 和逻辑 I/O 当作更稳的比较依据；延迟只解释同一环境下的相对趋势，不把它当成原生 SSD 的绝对性能。另外，`mean_ios` 是系统内部的逻辑 I/O，两套系统的页大小和计数方式可能不同，不能把它换成块设备物理 I/O，也不能单凭次数判断优劣。

实际执行的主要命令（工作目录 `~/fvs-course-lab`）：

```
bash scripts/run_debug_validation.sh
bash scripts/download_indexes.sh
bash scripts/run_formal_experiments.sh
```

下载脚本内部会调用 `check_indexes.sh`。正式脚本启动时会再次校验数据集和索引。两套索引校验均通过后才开始查询。

4. 实验结果与机制分析

下面单元格中的数是三轮中位数，括号内是最小值到最大值。Recall 和逻辑 I/O 的范围很窄；延迟范围明显更大。

4.1 Recall–Latency

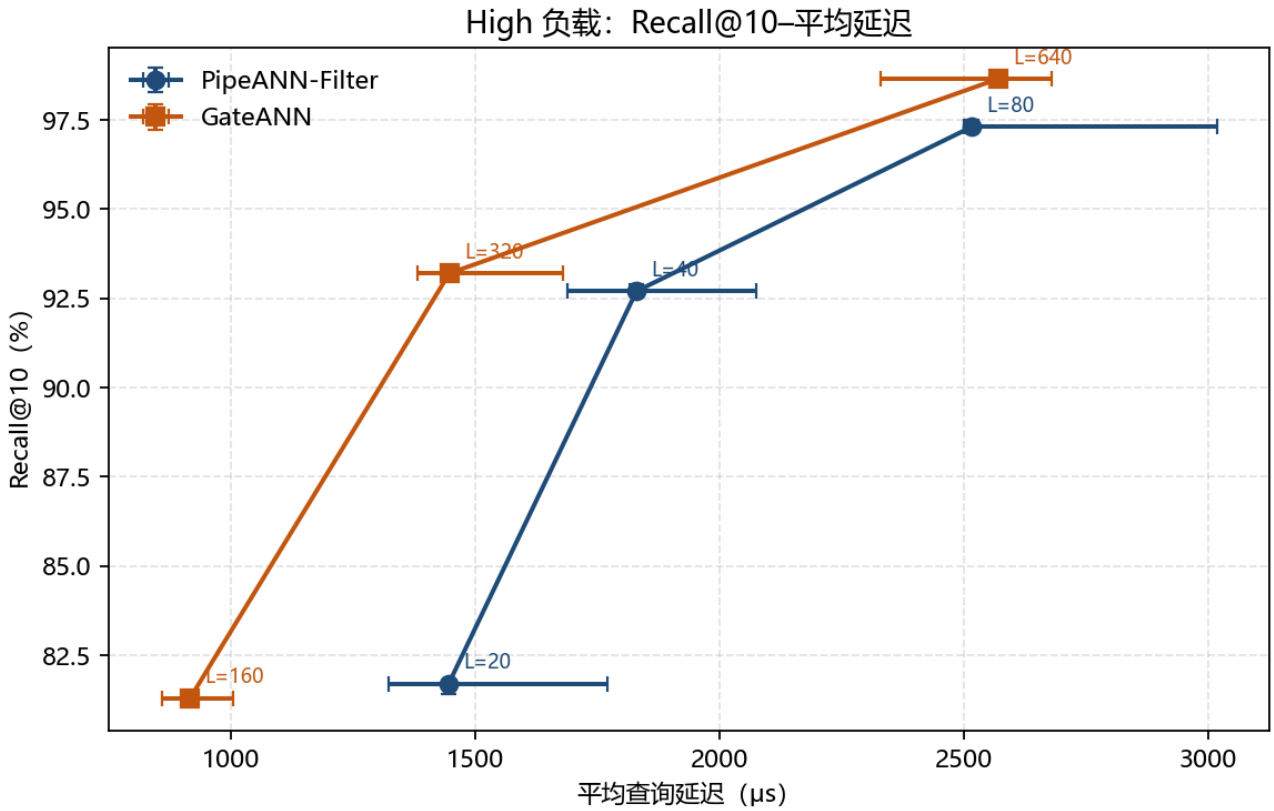


图 1　High 负载上的 Recall@10–平均延迟。点为三轮中位数，误差线为三轮最小到最大。横轴是 Query Phase 延迟。

系统	L	Recall@10（%）	Mean Latency（μs）	QPS	Mean I/O（次/查询）
PipeANN-Filter	20	81.70（81.40–81.80）	1445.8（1323.5–1770.0）	687.1（560.4–751.9）	100.0（99.0–100.6）
PipeANN-Filter	40	92.70（92.70–92.90）	1830.2（1688.1–2076.0）	543.5（479.3–589.7）	141.1（140.9–141.3）
PipeANN-Filter	80	97.30（97.20–97.50）	2515.9（2500.8–3017.6）	395.6（329.9–398.2）	229.2（228.5–229.4）
GateANN	160	81.30（81.25–81.35）	916.6（858.4–1005.0）	1091.0（995.0–1165.0）	24.66（24.64–24.68）
GateANN	320	93.20（93.20–93.20）	1447.2（1381.2–1680.7）	691.0（595.0–724.0）	46.75（46.74–46.75）
GateANN	640	98.65（98.65–98.65）	2570.7（2331.0–2681.0）	389.0（373.0–429.0）	88.94（88.93–88.94）

测量结果：在 High 上，两套系统的搜索预算从低档增到高档时，Recall、延迟和逻辑 I/O 一起上升。PipeANN-Filter 的 Recall 从 81.70% 升到 97.30%，逻辑 I/O 从 100.0 次升到 229.2 次，延迟中位数从 1445.8 μs 升到 2515.9 μs。GateANN 的 Recall 从 81.30% 升到 98.65%，逻辑 I/O 从 24.66 次升到 88.94 次，延迟中位数从 916.6 μs 升到 2570.7 μs。这是直接测得的事实。延迟的轮次波动大于 Recall 和 I/O，和第 3 节的页缓存现象一致。

在 Recall 接近的点上比较：L=20 的 PipeANN-Filter（Recall 81.70%，延迟 1445.8 μs，I/O 100.0）对 L=160 的 GateANN（81.30%，916.6 μs，24.66）；L=40 的 PipeANN-Filter（92.70%，1830.2 μs，141.1）对 L=320 的 GateANN（93.20%，1447.2 μs，46.75）。这两对点上 GateANN 的延迟更低，逻辑 I/O 大约只有 PipeANN-Filter 的四分之一到三分之一。到最高档时，PipeANN-Filter L=80 的延迟 2515.9 μs、Recall 97.30%，GateANN L=640 的延迟 2570.7 μs、Recall 98.65%。这里延迟已经接近，但 Recall 并不相同，不能写成同等质量下的加速比。GateANN 在该运行点上用相近延迟达到了更高 Recall，同时逻辑 I/O 仍少很多（88.94 对 229.2）。

内部指标和机制：PipeANN-Filter 在 High 的三档 L 上，200 条查询全部走 In-filter，Pre-filter 和 Post-filter 都是 0。假阳性中位数随 L 从 8.6 增到 12.5 再增到 19.2，与 I/O 上升方向一致。这符合 In-filter 用近似成员判断换取继续扩展：预算越大，推测访问越多，假阳性和读盘一起变多，Recall 也更高。GateANN 的 FilterSkips 从 143.16 增到 280.25 再增到 557.07，Tunnel 时间从 242 μs 增到 445 μs 再增到 808 μs，Process 时间从 258 μs 增到 425 μs 再增到 771 μs。预算变大时，它跳过的无效候选和隧道开销一起增加，所以延迟上升并不只来自那几十次逻辑 I/O。I/O 时间中位数只在 131–148 μs 间变化，远小于 Tunnel 和 Process 的增幅。因此在这份已经偏热的页缓存上，High 负载的延迟增量主要是计算和隧道，而不是读盘等待。这是结合计时字段作出的推断；它不能外推到冷缓存下的物理 SSD。

`mean_ios` 适合看同一系统随 L 和工作负载的变化。跨系统比较绝对值时，两套实现的 I/O 页和统计方式可能不同，本文只把它当作机制证据，不单独用它给系统排名。

4.2 Selectivity–Performance

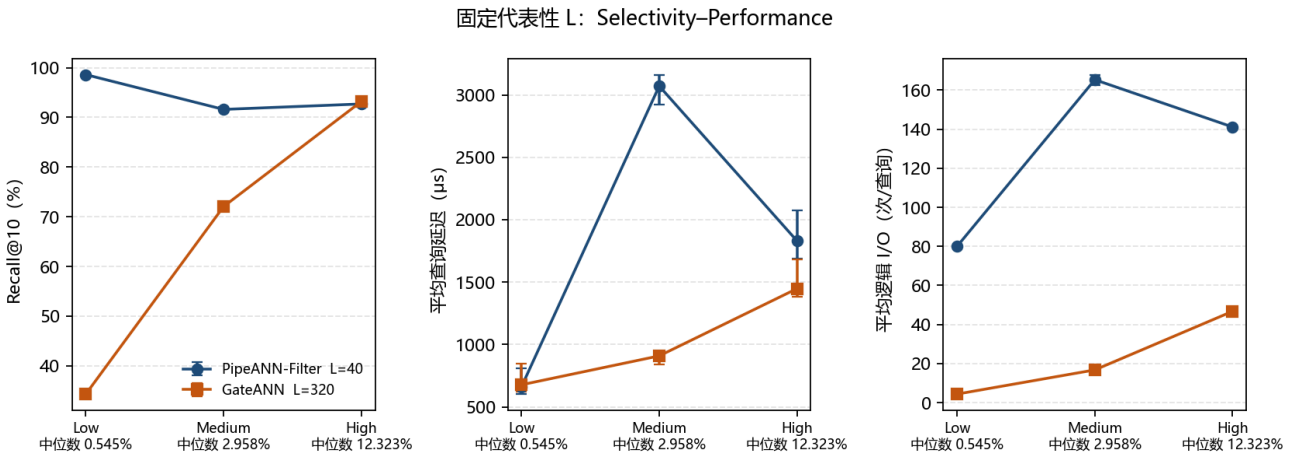


图 2　固定 PipeANN-Filter L=40、GateANN L=320。横轴三档的选择率中位数分别为 0.545%、2.958% 和 12.323%。点为三轮中位数，误差线为三轮范围。

系统	Bucket	Selectivity 范围/中位数 (%)	Recall@10 (%)	Mean Latency (μs)	Mean I/O (次/查询)	关键内部指标
PipeANN-Filter	Low	0.250–0.851 / 0.545	98.60 (98.60–98.60)	660.3 (602.2–810.7)	80.0 (80.0–80.0)	Pre/In/Post 200/0/0，假阳性 0.0
PipeANN-Filter	Medium	1.765–4.995 / 2.958	91.60 (91.50–92.10)	3071.0 (2925.6–3162.5)	165.3 (162.4–167.9)	Pre/In/Post 0/200/0，假阳性 26.1
PipeANN-Filter	High	7.074–17.089 / 12.323	92.70 (92.70–92.90)	1830.2 (1688.1–2076.0)	141.1 (140.9–141.3)	Pre/In/Post 0/200/0，假阳性 12.5
GateANN	Low	0.250–0.851 / 0.545	34.30 (34.30–34.30)	676.6 (619.2–843.9)	4.38 (4.38–4.38)	Skips 320.69；Tunnel 410 μs；I/O 16 μs；Process 31 μs
GateANN	Medium	1.765–4.995 / 2.958	72.05 (72.05–72.05)	908.3 (840.3–916.6)	16.75 (16.75–16.75)	Skips 309.35；Tunnel 452 μs；I/O 42 μs；Process 117 μs
GateANN	High	7.074–17.089 / 12.323	93.20 (93.20–93.20)	1447.2 (1381.2–1680.7)	46.75 (46.74–46.75)	Skips 280.25；Tunnel 445 μs；I/O 131 μs；Process 425 μs

PipeANN-Filter 的内部指标写成 Pre/In/Post 查询数和平均假阳性。三轮路由计数没有波动，假阳性仅在小数位上变化。GateANN 的 Skips 和逻辑 I/O 三轮几乎不变；Tunnel、I/O time、Process 是中位数。

测量结果：过滤变强时，两套系统的表现不一样，而且都不是“越强越慢”这种单方向变化。

PipeANN-Filter 在 Low 上把 200 条查询全部路由到 Pre-filter，假阳性为 0，Recall 98.60%，延迟中位数只有 660.3 μs，逻辑 I/O 80.0。Medium 和 High 则全部走 In-filter。Medium 的 Recall 降到 91.60%，延迟升到 3071.0 μs，I/O 升到 165.3，假阳性 26.1，是三档里最慢的一档。High 仍走 In-filter，但假阳性降到 12.5，I/O 141.1，延迟 1830.2 μs，Recall 92.70%。所以 Low 明显更快，是因为路径从 In-filter 换成了 Pre-filter，这是测得的路由事实。Medium 比 High 更慢，则发生在同

一条 In-filter 路径上：更强的过滤伴随更多假阳性、更多逻辑 I/O 和更高延迟。结合 In-filter 用近似判断继续扩展的机制，一种解释是中等选择率下推测访问更不容易命中真正满足谓词的点。这是推断。三档使用不同真实标签，标签频率和向量分布也不同，不能把 Medium 与 High 的全部差距都归因于 Selectivity 这一个变量。

GateANN 没有换路径。选择率从 High 降到 Low 时，Recall 从 93.20% 降到 72.05% 再降到 34.30%，逻辑 I/O 从 46.75 降到 16.75 再降到 4.38，延迟中位数从 1447.2 µs 降到 908.3 µs 再降到 676.6 µs。FilterSkips 从 280.25 升到 309.35 再升到 320.69。Low 上 I/O 时间只有 16 µs，Process 时间 31 µs，Tunnel 时间仍有 410 µs，占了查询时间的大部分。也就是说，过滤变强以后无效读确实被跳过了，但 Recall 掉得很快；剩下的时间主要花在隧道上，而不是 SSD 读。固定的 L=320 是按 High 校准的代表性预算。在 Low 上它已经几乎不再读盘，却只达到 34.30% 的 Recall。由此推断，此时的短板是过滤后图的可达性和搜索预算，不是读盘次数。Low 的 Recall 在三轮里都是 34.30%，不是某一次运行失败。

4.3 资源 Trade-off

系统	索引大小	查询峰值 RSS (MiB)	关键额外结构	获得的收益
PipeANN-Filter	8.43 GiB	138.5（138.4–138.7）	SSD 上的大索引；查询时 DRAM 足迹较小	Low 上 Pre-filter 同时拿到高 Recall 和低延迟；High 上 In-filter 把 Recall 推到 97% 以上
GateANN	486 MiB	408.4（408.3–408.6）	DRAM 中的标签矩阵约 49 MB，FullAdj 约 0.123 GB	相近 Recall 下逻辑 I/O 少很多；索引比 PipeANN-Filter 小一个数量级

索引大小在同一系统的各行中一致，上表各报一个值。峰值 RSS 按每次查询进程统计：High 的三档 L 共用同一次进程，因此没有按 L 重复计算，每套系统用 3 轮 × 3 个 bucket 共 9 个进程取中位数。`peak_rss_kb` 是整个查询子进程生命周期的峰值，不能把两套系统相差的约 270 MiB 全部算到 FullAdj 上。日志里的 FullAdj 约 126 MiB、标签矩阵约 49 MB，合计约 175 MiB，能解释差额的一大部分，其余还有 PQ 表、运行时缓冲和加载方式的差别。

资源上的交换是：GateANN 用大约 408 MiB 的查询内存和一块 486 MiB 的索引，在 High 的相近 Recall 点上把逻辑 I/O 降到 PipeANN-Filter 的大约三分之一，延迟也更低。PipeANN-Filter 的查询内存只有约 139 MiB，但索引有 8.43 GiB，In-filter 路径的逻辑 I/O 更高。这块额外的 SSD 空间并没有在本次 High 负载上换成更少的逻辑 I/O。反过来，GateANN 的内存结构在 Low 上没有换成 Recall：I/O 已经降到 4.38 次，Recall 仍只有 34.30%。所以 FullAdj 换来的是少读盘，不是在所有选择率上都更准。

4.4 关键现象与瓶颈总结

现象一：High 上增加搜索预算时，两套系统的 Recall、延迟和逻辑 I/O 同步上升；在 Recall 约 82% 和约 93% 的两个运行点上，GateANN 的逻辑 I/O 和延迟都低于 PipeANN-Filter。PipeANN-Filter 的内部证据是三档全部走 In-filter，假阳性随 L 从 8.6 增到 19.2。GateANN 的内部证据是 FilterSkips、Tunnel 时间和 Process 时间随 L 明显增加，而 I/O 时间几乎不动。对应机制分别是推测式 In-filter，以及 Pre-I/O Filtering 加 Graph Tunneling。由此判断，在本机这份热缓存上，High 负载继续加预算的主要代价已经转到计算和隧道，而不是读盘等待。冷缓存下读盘是否重新成为延迟主体，本次没有测量。

现象二：过滤加强后，PipeANN-Filter 在 Low 上切到 Pre-filter，Recall 保持 98.60% 且延迟降到 660.3 μs；GateANN 不换路径，Low 上 Recall 降到 34.30%，同时逻辑 I/O 只剩 4.38，时间主要在 Tunnel（410 μs），不在 I/O（16 μs）。对应机制是：Pre-filter 先收集合条件的候选，强过滤时候选少，所以又快又准；GateANN 用跳过读盘来省 I/O，靠固定预算的隧道恢复连通性，预算是按 High 取的 L=320。由此判断 GateANN 在 Low 上的瓶颈是过滤后的可达性和搜索预算，不是 SSD 读次数。分桶还混有标签和向量分布差异，所以这是该工作负载上的瓶颈判断，不是对 Selectivity 单一变量的因果结论。

5. 优化方向

下面继续分析现象二里的 GateANN。没有另做补充实验，直接使用任务二数据。

Observation

固定 GateANN L=320 时，Low（选择率中位数 0.545%）的 Recall@10 为 34.30%，High（12.323%）为 93.20%。同一组运行里，Low 的平均逻辑 I/O 为 4.38 次/查询，FilterSkips 为 320.69，I/O 时间中位数 16 μs，Tunnel 时间中位数 410 μs，Process 时间中位数 31 μs。三轮 Recall 和逻辑 I/O 没有波动。

Mechanism

GateANN 在发起 SSD 读之前按谓词丢弃邻居，并用内存中的 FullAdj 做 Graph Tunneling，以恢复过滤后的图可达性（`include/utils/full_adj_index.h`，查询入口 `tests/search_disk_index_yfcc.cpp`）。FilterSkips 高、逻辑 I/O 低，说明 Pre-I/O Filtering 已经生效。Tunnel 时间远大于 I/O 时间，说明剩余开销在隧道遍历，不在读盘。

Bottleneck

已有证据支持的结论是：在 Low 上，读盘次数已经不是 Recall 上不去的原因。尚未验证的假设是：把 L 或隧道深度按选择率加大以后，Recall 会明显回升，并且新增时间主要出现在 Tunnel 而不是 I/O。另一种可能是过滤后的子图本身缺少长程连接，只加大 L 也走不远。现有三档 L 都测在 High 上，不能直接证明 Low 上加预算有效。

Optimization

在查询阶段按选择率或按已观察到的 FilterSkips 调整搜索预算，而不是对所有桶固定 L=320。选择率低、跳过次数已经接近 L 时，增大 L（或等价地增加隧道扩展轮数）；选择率高、I/O 时间开始上升时，保持或减小预算。不改 FullAdj 的邻居数，也不改索引格式。

Expected Benefit

预期改善的是 Low，以及可能的 Medium：Recall@10 上升。High 上如果预算不增加，延迟和 I/O 应保持接近本次 L=320 的结果。适用工作负载是强过滤、当前代表性 L 下逻辑 I/O 已经很低但 Recall 明显不足的查询。

Trade-off

Low 上隧道时间已经是大头。加大 L 会增加 CPU 上的隧道和候选处理时间，延迟会高于现在的 676.6 μs，也可能多出一些逻辑 I/O。DRAM 和 SSD 索引大小不变，构建时间不变。实现上要避免把

High 的延迟也一起拉高，所以预算必须随工作负载变化，不能全局改成一个更大的常数。

Validation Plan

对照组是本次 Low、L=320、threads=1、beamwidth=8、K=10、FullAdj neighbors=32 的结果（Recall 34.30%，逻辑 I/O 4.38，Tunnel 410 µs）。自变量只改 GateANN 在 Low 上的 L，取 640、1280、2560。控制变量包括同一台机器、同一正式索引、同一 Low 查询集、相同 I/O backend 和线程数。每档重复 3 次。指标为 Recall@10、mean latency、mean I/O、Tunnel time 和 I/O time。若 Recall 升到 80% 以上，且新增延迟主要由 Tunnel time 解释、mean I/O 仍明显低于 PipeANN-Filter 在 Low 上的 80 次，则支持该优化。若 L 增大后 Tunnel time 近似线性上升，而 Recall 仍停在 40% 附近，则否定“只加预算就够”的假设，说明需要在建索引时增加过滤后的长程边，而不是只改查询预算。

6. 实验总结与结论

两套系统最重要的差别是过滤放在什么地方。PipeANN-Filter 在搜索过程中按估计代价选择 Pre-filter、In-filter 或 Post-filter。GateANN 在读盘前过滤，并用内存里的 FullAdj 做隧道，换取更少的无效 I/O 和额外的 DRAM。

本次最有代表性的结果有两个。第一，High 上 Recall 约 93% 时，GateANN（L=320）延迟中位数 1447.2 µs、逻辑 I/O 46.75 次，PipeANN-Filter（L=40）为 1830.2 µs、141.1 次。第二，过滤很强时两者走向相反：PipeANN-Filter 改走 Pre-filter，Recall 仍有 98.60%；GateANN 的逻辑 I/O 降到 4.38 次，Recall 只有 34.30%，时间主要花在隧道上。

PipeANN-Filter 更适合强过滤、可以走 Pre-filter 的工作负载，以及希望查询内存保持在约 140 MiB 的场景，代价是约 8.43 GiB 的索引和 In-filter 上更高的逻辑 I/O。GateANN 更适合过滤不算太强、希望少读盘并且索引只有几百 MiB 的场景；在本次 Low 负载上，固定 L=320 的 Recall 不够用。

优化方向的核心判断是：GateANN 在强过滤下的短板不是再减少 I/O，而是按选择率加大隧道和搜索预算。这个判断有 I/O、FilterSkips 和 Tunnel 时间的证据，但还没有用更大的 L 在 Low 上验证。

结论的界限也需要说明。实验在 WSL2 虚拟磁盘上完成，页缓存是热的，200 条查询又很短，所以绝对延迟和 QPS 不能代表原生 SSD，也不适合拿去和其他机器比。逻辑 I/O 不是物理 I/O，跨系统只能结合 Recall 一起看。三档桶使用不同真实标签，桶间差异不能全部写成 Selectivity 的因果效应。在这些界限之内，这次实验说明：SSD 上的过滤向量检索要同时决定过滤发生在读盘前还是搜索中、过滤之后图还通不通，以及为此付出的是磁盘空间、内存还是 CPU。

原始结果保留在 `course_runs/experiments/20260930-072449/` 的 `results.csv`、`environment.txt`，以及 `course_runs/results/` 下各次运行的 `command.json` 和 `run.log`，供课程抽查。报告不附这些文件。